

Lecture 11: Memory and Control flow

CMPE 120: Computer Organization & Architecture

Olivia Weng

Logistics

- Reminder: check-in with professor is 1% of your final grade
 - Sign up [link](#)
- HW2 due October 1st

RISC-V instruction format

DP4: Good design demands **good compromises**.

- 4 types of instruction formats

- R-type: register

- I-type: immediate — *constants numbers*

- S/B-type: Store/Branch

- U/J-type: Upper-Immediate/Jump

7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
funct7	rs2	rs1	funct3	rd	op
imm _{11:0}		rs1	funct3	rd	op
imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op
imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	op
imm _{31:12}				rd	op
imm _{20,10:1,11,19:12}				rd	op
20 bits				5 bits	7 bits

R-Type

I-Type

S-Type

B-Type

U-Type

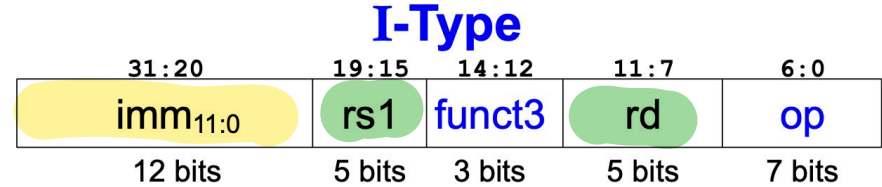
J-Type

I-type instruction encoding

- Immediate-type

- 3 operands

- rs1: source register
- rd: destination register
- imm: 12-bit two's complement immediate



- op: operation code (opcode)

- funct3:

- the function combined with opcode tells machine exactly what operation to perform

Example instruction encoding

- $x8 = x9 + 12$

`addi x8, x9, 12`

I-Type

31:20	19:15	14:12	11:7	6:0
imm _{11:0}	rs1	funct3	rd	op
12 bits	5 bits	3 bits	5 bits	7 bits

imm=12,

0000 0000 1100

rs1=9,

0100 1

funct3=0,

000

rd=8,

0100 0

op=19

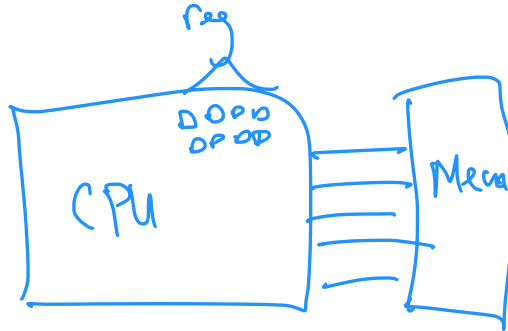
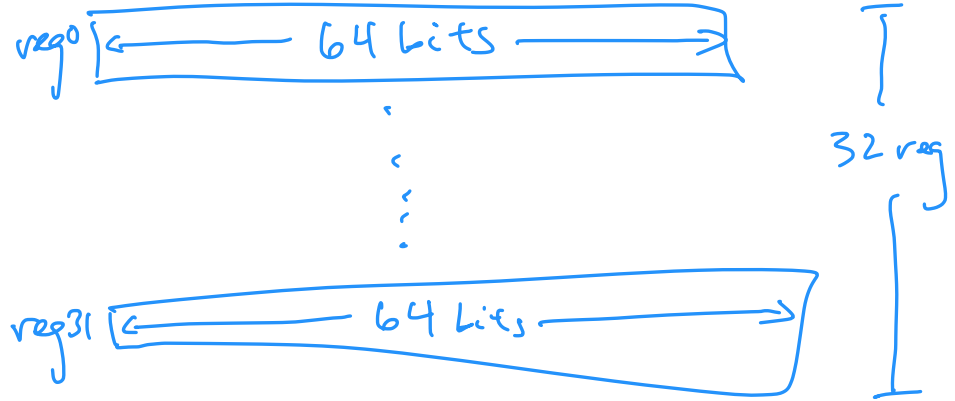
001 0011

00000000110001001000010000010011

0x00C48413

Accessing operands

- Operands are in 1 of 2 places:
 - registers (32 registers)
 - each register stores 64 bits
 - memory (2^{64} locations)

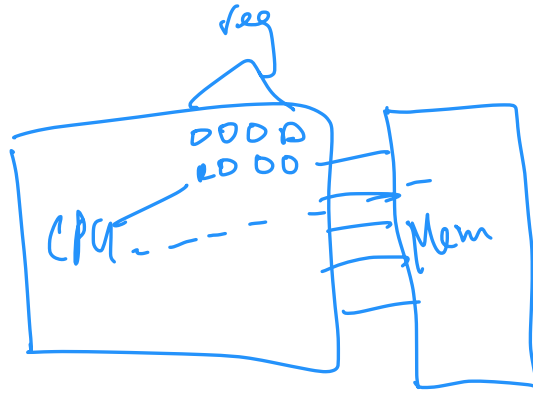


Accessing operands

- Operands are in 1 of 2 places:
 - registers (32 registers)
 - each register stores 64 bits
 - memory (2^{64} locations)
- Textbook uses RV64 version of RISC-V
 - RV64 is a 64-bit architecture because it operates on 64-bit data
 - RISC-V word = 32 bits
 - RISC-V doubleword = 64 bits

Accessing operands

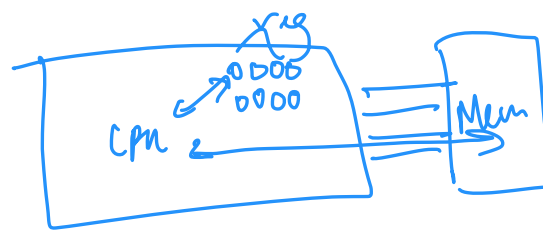
- Registers are
 - easy to specify
 - close to the processor (fast access!)



Accessing operands

- Registers are
 - easy to specify
 - close to the processor (fast access!)
- Goal: Use registers **whenever** possible
- Idea: load-store architectures
 - normal **arithmetic** instructions **only** access **registers**
 - only access **memory** with explicit loads and stores

Accessing operands



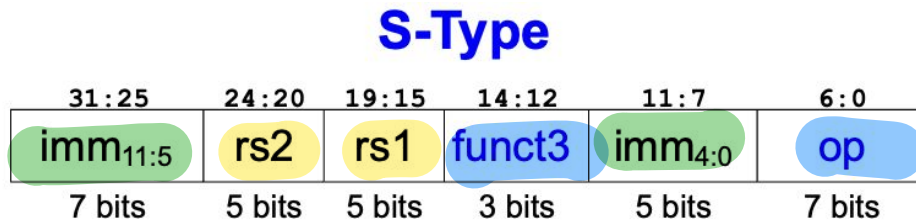
2^{64} memory locations
32 reg

- In each column, we have which (reg or mem) is better. Which row is correct?

	Faster access	Fewer bits to specify	More locations
A	Mem	Mem (64 bits)	Reg
B	Mem	Reg (5 bits)	Mem
C	Reg	Mem	Reg
D	Reg	Reg	Mem
E	None of the above		

S-Type instruction encoding

- Store-type
- 3 operands
 - rs1: source register 1
 - rs2: source register 2
 - imm: 12-bit two's complement immediate



- op: operation code (opcode)
- funct3:
 - the function combined with opcode tells machine exactly what operation to perform

Example instruction encoding

memory location = $x19 - 6$

- Store x7's data at address x19-6

S-Type

31:25	24:20	19:15	14:12	11:7	6:0
imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

sw x7, -6(x19)

store word

imm=1111 111, rs2=7, rs1=19, funct3=2, imm=11010, op=35
 1111 111 0 0111 1001 1 010 1101 0 010 0011

11111110011110011010110100100011

0xFE79AD23

RISC-V is a **load-store** architecture

- Forces heavy dependence on registers (good for modern CPUs!)
 - Tradeoff: **fast** implementation but **more** instructions

RISC-V is a **load-store** architecture

- Forces heavy dependence on registers (good for modern CPUs!)
 - Tradeoff: **fast** implementation but **more** instructions

- Can do

`add s3, s4, s5`

`sw s3, 8(s0)`

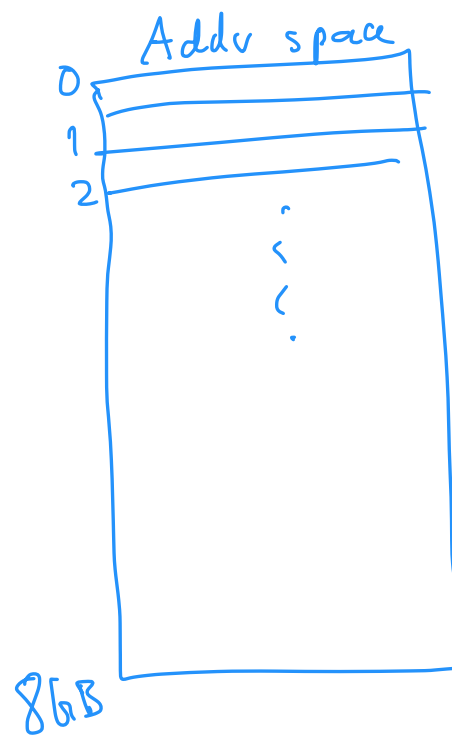
- Can't do

`add s1, s4, 8(s0)`

memory address

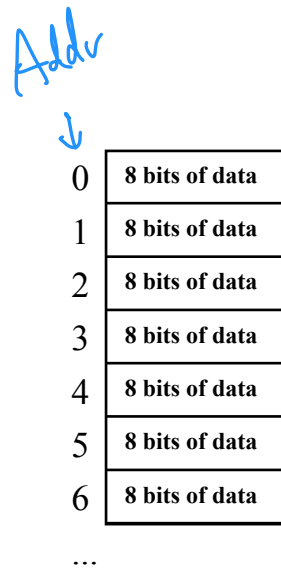
What does **memory** look like?

- Viewed as large 1-dimensional **array**
- Memory **address** is an index into the array



What does **memory** look like?

- Viewed as large 1-dimensional **array**
- Memory **address** is an index into the array
- RISC-V is **byte-addressable**
 - Index (address) points to a **byte** of memory



Memory access are often required to be word-aligned

→ wires (lots!)



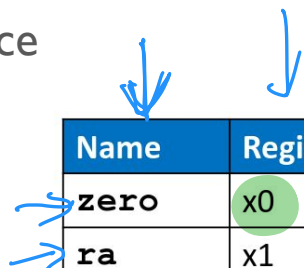
Addr

- Because of how busses and memory work
- Most data items use words or doublewords
 - Recall: RISC-V word = 32 bits or 4 bytes
- Words are aligned!

0	32 bits of data
4	32 bits of data
8	32 bits of data
12	32 bits of data

RISC-V register set

- For reference



Name	Register Number	Usage
 zero	 x0	<u>Constant value 0</u>
 ra	<u>x1</u>	<u>Return address</u>
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporaries
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments / return values
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	 x28-31	Temporaries

Registers

- Registers can use either **name** (i.e., zero, ra) or **number** x0, x1, etc.
 - Name is preferred

Registers

- Registers can use either **name** (i.e., zero, ra) or **number** x0, x1, etc.
 - Name is preferred
- Certain registers used for **specific** purposes
 - zero always holds the constant value 0
 - **saved registers** (s0 - s11) hold variables
 - **temporary registers** (t0 - t6) hold intermediate values during large computation

RISC-V addressing modes & syntax

- **register direct** (R-type)

C code	RISC-V assembly code
<code>a = b + c;</code>	<code>add a, b, c</code>

RISC-V addressing modes & syntax

- **register direct** (R-type)

C code	RISC-V assembly code
<code>a = b + c;</code>	<code>add a, b, c</code>

- **immediate** (I-type)

C code	RISC-V assembly code
<code>int a = 5;</code> <code>int b = a + 6;</code>	<code># s0 = a, s1 = b</code> <code>addi s0, zero, 5</code> <code>addi s1, s0, 6</code>

mv rd, rs

RISC-V addressing modes & syntax

- **register direct** (R-type)

C code	RISC-V assembly code
<code>a = b + c;</code>	<code>add a, b, c</code>

- **immediate** (I-type)

C code	RISC-V assembly code
<code>int a = 5; int b = a + 6;</code>	<code># s0 = a, s1 = b addi s0, zero, 5 addi s1, s0, 6</code>

- **Base addressing**
 - base + immediate

C code	RISC-V assembly code
<code>array[i]</code>	<code>lw t1, 8(s0) # lw dest, offset(base)</code>

Handwritten notes:

- load word* (pointing to `lw`)
- s0 + 8* (pointing to `8(s0)`)
- addr in this reg.* (pointing to `s0`)

RISC-V addressing modes & syntax

- **PC-relative**: go to instruction some distance away relative to PC
 - **PC**: program counter that keeps track of current instruction's *addr*

RISC-V addressing modes & syntax

- **PC-relative**: go to instruction some **distance away relative** to PC
 - **PC**: **program counter** that keeps track of current instruction
- Used in branch (B-type) and jump (J-type) instructions
 - Rather than goto next instruction, goto instruction further away
 - When would that be useful?

RISC-V addressing modes & syntax

- **PC-relative**: go to instruction some **distance away relative** to PC
 - **PC**: **program counter** that keeps track of current instruction

- Used in branch (B-type) and jump (J-type) instructions
 - Rather than goto next instruction, goto instruction further away
 - When would that be useful?

if [condition]
 ↳ branch 1
else
 ↳ branch 2

- Useful for **control flow** (e.g., if-else, loops, function calls and return, etc.)

Control flow

- Control flow describes **how program executes**

- Jumps
- Procedure calls
- Conditional branch
 - if-then-else logic, loops, etc.

- Control flow specifies

- **Condition** (if any) under which jump or branch is taken
- If taken, the **location** to read the **next instruction** from (i.e., the **target**)

↳ branch target

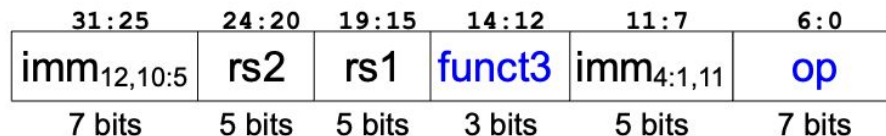
B-Type instruction encoding

- Branch-type

- 3 operands

- rs1: source register 1
- rs2: source register 2
- imm: upper 12 bits of 13-bit two's complement immediate (address offset)
 - Implicit 0-bit as LSB

B-Type



- op: operation code (opcode)

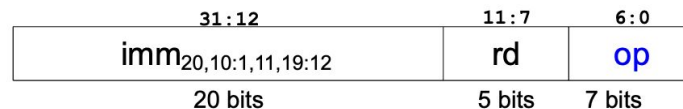
- funct3:

- the function combined with opcode tells machine exactly what operation to perform

J-type instruction encoding

riscv instr are 32 bits long

J-Type



- **Jump-type**
 - Used for jump-and-link instruction (jal)
 - jal: jump to PC-relative address and save next PC (i.e., PC +4) in destination register (typically ra)
- **2 operands**
 - rd: **destination** register
 - imm: Upper 20 bits of 21-bit immediate
 - There's an implicit 0-bit as LSB
- op: operation code (**opcode**)

Demo

gemu

gdb

- We need to use a RISC-V emulator to run RISC-V programs

```
$ riscv64-linux-gnu-gcc -g -O0 -static -o hello hello.c
```

```
$ qemu-riscv64 -g 1234 ./hello & gdb-multiarch ./hello
```

```
$ (gdb) target remote :1234
```

```
$ (gdb) set architecture riscv:rv64
```